

Pasang di VPS

Satu VPS cukup untuk semuanya: dashboard, mesin WhatsApp, database, dan berkas lampiran. Tidak ada layanan tambahan.

Sebelum mulai

VPS di Jakarta atau Singapura. Pemakainya pemilik usaha di Indonesia, dan server di Eropa terasa jauh lebih lambat di dashboard. Mulai dari 2 GB memori. Aplikasinya sendiri sekitar 400 MB tanpa nomor tersambung, dan tiap nomor WhatsApp menambah lagi.

Dua DNS record ke IP server, plus `www` kalau mau:

```
A    @    <ip-server>
A    app  <ip-server>
A    www  <ip-server>
```

Nomor WhatsApp khusus usaha, bukan nomor pribadi.

1. Siapkan servernya

Ada skrip untuk seluruh langkah ini di `pasang-server.sh` di akar repo. Karena kodenya belum ada di server pada titik ini, salin tempel isinya ke terminal, atau kerjakan perintah di bawah satu per satu. Skripnya aman dijalankan berulang: tiap bagian memeriksa dirinya dulu.

Skripnya **tidak** mematikan login SSH pakai password, dan itu disengaja. Lihat alasannya di akhir bagian ini.

Masuk sebagai root lewat SSH, lalu:

```
apt update && apt install -y curl git
curl -fsSL https://deb.nodesource.com/setup_20.x | bash -
apt install -y nodejs
npm install -g pm2
```

Caddy ikut petunjuk resminya di caddyserver.com/docs/install.

Tutup porta yang tidak dipakai. Porta 4000 sengaja TIDAK dibuka: di baliknya ada kemampuan mengirim WhatsApp atas nama pelangganmu dan membaca lampiran mereka.

```
ufw allow 22 && ufw allow 80 && ufw allow 443 && ufw --force enable
```

Tambah swap 2 GB. Di VPS 2 GB, `npm run build` sering mati sendiri di tengah jalan dengan pesan `killed`, dan itu terbaca seperti bug padahal cuma kehabisan memori.

```
fallocate -l 2G /swapfile && chmod 600 /swapfile && mkswap /swapfile && swapon /swapfile && echo '/swap
```

Keraskan SSH-nya. Ini pengaman terpenting di seluruh panduan, dan bukan soal kode. Di mesin ini ada database pelanggan tanpa sandi berkas `DAN` folder `data/wa-sessions` yang isinya kunci sesi WhatsApp tiap pelangganmu. Siapa pun yang bisa menyalin folder itu bisa mengirim WhatsApp atas nama toko pelangganmu, dan tidak ada satu pun pengaman di dalam aplikasi yang menolong kalau itu terjadi.

```
sed -i 's/^##*PasswordAuthentication.*/PasswordAuthentication no;/ s/^##*PermitRootLogin.*/PermitRootLogi
```

Jalankan itu HANYA setelah kunci SSH-mu benar-benar bisa dipakai masuk, kalau tidak kamu mengunci diri sendiri di luar. Tambah `fail2ban` sekalian.

2. Ambil kodenya

Repo-nya ada di GitHub sejak 9 Agustus 2026, jadi `git clone` yang dipakai, bukan lagi `rsync` dari laptop. Ini lebih baik dan bukan cuma lebih pendek: yang sampai di server persis versi yang tercatat, `git pull` tahu apa yang berubah, dan tidak ada berkas nyasar dari laptop yang ikut terkirim.

```
apt-get install -y git
git clone https://github.com/13kailouis/palwiseAICRM.git /opt/palwise
cd /opt/palwise && npm install && cp .env.example .env
```

Yang TIDAK ikut ter-clone, dan itu memang benar: `.env` dan seluruh folder `data/`, keduanya ada di `.gitignore`. Jadi kunci laptopmu tidak pernah menimpa kunci server, dan database laptopmu tidak pernah menimpa database pelanggan. Dua hal itu diisi sendiri di server, lihat langkah 3 dan 4.

Repo-nya publik, dan yang perlu diingat dari itu cuma satu: apa pun yang dikomit bisa dibaca semua orang, termasuk nanti. Jangan pernah `git add -f .env`, dan periksa dulu sebelum menambah berkas yang memuat angka atau kunci sungguhan. Sampai 9 Agustus 2026 riwayatnya sudah diperiksa dan bersih: `.env` tidak pernah masuk, dan tidak ada satu pun kunci API di seluruh riwayat.

Kalau nanti repo-nya dijadikan privat, `git clone` butuh kunci deploy. Buat kuncinya di server (`ssh-keygen`), lalu tambahkan yang `.pub` sebagai Deploy Key di setelan repo GitHub.

3. Isi .env, dan ini bagian yang paling menentukan

Jangan salin .env laptop ke server

Menyalinnya kelihatan lebih cepat, dan justru itu masalahnya. .env laptopmu memuat MIDTRANS_PRODUCTION kosong (artinya sandbox), dua alamat situs kosong, serta AUTH_SECRET dan INTERNAL_TOKEN milik laptop. Kalau disalin lalu ada satu yang lupa diubah, **gagalnya diam-diam**: pembayaran jalan di sandbox padahal pelanggan sungguhan sudah masuk, dan tidak ada satu pun galat yang memberi tahu.

Mulai dari .env.example justru gagalnya BERISIK, dan itu yang kita mau. Lupa INTERNAL_TOKEN bikin worker menolak jalan. Lupa AUTH_SECRET bikin halaman masuk melempar galat. Lupa MIDTRANS_SERVER_KEY bikin tombol paket berbayar menolak dengan sopan. .env.example memuat SEMUA kunci yang ada di .env laptop, jadi tidak ada yang ketinggalan. Kalau nanti menambah kunci baru, tambahkan di dua-duanya.

```
cd /opt/palwise && cp .env.example .env && nano .env
```

Pakai nano di server, jangan menulisnya di Notepad lalu diunggah. Notepad menyisipkan tanda BOM di awal berkas. Sekarang tidak berbahaya karena baris pertama .env memang komentar, tapi kalau suatu hari baris pertamanya sebuah variabel, namanya jadi ikut membawa BOM, dotenv tidak menemukannya, dan tidak ada galat apa pun. **Biarkan baris pertama tetap komentar**, itu jaring pengamanannya.

Yang mana disalin, yang mana dibuat baru

Kunci	Di server
GEMINI_API_KEY , RESEND_API_KEY , MIDTRANS_SERVER_KEY	Salin nilainya dari .env laptop
AUTH_SECRET , INTERNAL_TOKEN	Buat baru , openssl rand -base64 32
NEXT_PUBLIC_SITE_URL , NEXT_PUBLIC_APP_URL	Isi alamat sungguhannya
MIDTRANS_PRODUCTION	on
FOUNDER_EMAILS	Emailmu
OPENAI_API_KEY , ANTHROPIC_API_KEY	Biarkan kosong
Sisanya	Biarkan seperti bawaan .env.example

NODE_ENV tidak ada di .env dan memang tidak perlu ditambahkan. ecosystem.config.cjs sudah menyetelnya production untuk kedua proses, dan nilai yang sudah ada di lingkungan tidak ditimpa dotenv. Jangan mencarinya di .env.example , dia memang tidak di situ. Yang dinyalakan

`NODE_ENV=production` : pesan galat berubah jadi bahasa manusia (bukan "jalankan npm run dev"), dan kegagalan kirim email berhenti dimaafkan.

Yang di bawah ini yang perlu kamu pikirkan satu per satu.

```
INTERNAL_TOKEN=<huruf acak panjang>
```

Buat dengan `openssl rand -base64 32` . Ini satu-satunya yang memisahkan mesin WhatsApp dari siapa pun yang bisa menjangkau portnya, dan di dalamnya ada kemampuan mengirim pesan atas nama pelangganmu. **Worker akan MENOLAK JALAN** kalau ini masih nilai bawaannya dan

`NODE_ENV=production` .

```
RESEND_API_KEY=<kunci dari resend.com>
```

Kalau kosong, orang yang lupa password dibalas "cek emailmu ya" lalu menunggu email yang tidak akan pernah datang, dan alamat email itu satu-satunya jalan mengembalikan akunnya. Jatah gratis Resend 3.000 email sebulan, jauh di atas kebutuhan.

```
AUTH_SECRET=<huruf acak panjang, yang lain lagi>
```

Kunci yang menjaga sesi login. Kodenya cuma menolak kalau kolomnya KOSONG, jadi nilai bawaan `ganti-dengan-huruf-acak-yang-panjang-juga` di `.env.example` akan lolos diam-diam, dan nilai itu tertulis di dalam repo. Buat yang baru di server, jangan pakai yang sama dengan laptop.

```
GEMINI_API_KEY=<kunci dari aistudio.google.com/apikey>
NEXT_PUBLIC_SITE_URL=https://palwise.id
NEXT_PUBLIC_APP_URL=https://app.palwise.id
EMAIL_FROM=Palwise <noreply@palwise.id>
```

Dua alamat itu menentukan sitemap, canonical, dan pengalihan antara halaman jualan dan dashboard. Kalau kosong, sitemap keluar kosong dan pengalihannya mati.

```
MIDTRANS_SERVER_KEY=<Server Key dari dashboard.midtrans.com>
MIDTRANS_PRODUCTION=on
```

Ambil di dashboard Midtrans, menu Settings > Access Keys, dan pastikan pemilih **Environment** di kiri atas ada di **Production**. Kunci sandbox berawalan `SB-` , kunci production tidak.

`MIDTRANS_PRODUCTION` bawaannya kosong yang berarti sandbox, dan itu memang disengaja: sandbox di server sungguhan cuma bikin pembayaran tidak jadi, sedangkan production di laptop menagih uang sungguhan waktu kamu sedang mencoba-coba. Kalau kunci dan mode tidak cocok, halaman Paket & pemakaian mengatakannya dengan kalimat lengkap, bukan 401 kosong.

Kalau `MIDTRANS_SERVER_KEY` dikosongkan, tombol paket berbayar menolak dengan sopan dan menyuruh menghubungi kamu. Yang TIDAK terjadi: paketnya naik tanpa dibayar.

```
FOUNDER_EMAILS=emailkamu@gmail.com
```

Siapa yang boleh membuka `/app/founder`, dipisah koma. **Kosong berarti halamannya tidak ada sama sekali** (dijawab 404, bukan "kamu tidak punya akses", supaya keberadaannya tidak ikut bocor). Isinya cuma hitungan: pendaftar, aktivasi, MRR, tagihan menunggu, dan kotak masukan. Isi chat pelanggan sengaja tidak pernah ditampilkan di situ.

Yang TIDAK perlu kamu sentuh: `GEMINI_MODEL`. Bawaannya `gemini-3.1-flash-lite`, dan itu keputusan harga, bukan kualitas. Model kelas di atasnya menagih \$9,00 per 1 juta token keluaran, yang jatuhnya Rp 153 per balasan, sementara paket Growth dijual Rp 33 per balasan. Naik kelas berarti setiap balasan rugi. Kalau suatu hari mau diganti, hitung ulang harga paketnya dulu dan jalankan `npm run uji:model`.

4. Bangun dan siapkan database

Buat folder `data/` dulu, dan ini WAJIB. Foldernya ada di `.gitignore`, jadi sesudah `git clone` dia belum ada di server. Dan SQLite tidak bisa membuat folder sendiri: `npm run db:push` akan gagal dengan "unable to open database file", pesan yang tidak menyebut folder sama sekali dan bikin orang mencari kesalahan di tempat yang salah.

`data/log` juga perlu ada sebelum PM2 jalan. Kalau tidak, catatan prosesnya tidak ikut tertulis, dan itu justru yang kamu butuh baca waktu ada yang rusak jam 2 pagi.

```
cd /opt/palwise && mkdir -p data/log data/media data/wa-sessions data/cadangan
```

```
npm run db:push
npm run build
npm run logo
```

`npm run logo` membuat favicon, ikon, dan kartu bagikan dari `aset/logo.jpg`.

Jangan jalankan `npm run db:seed` di server. Itu data contoh untuk laptop.

5. Nyalakan

```
pm2 start ecosystem.config.cjs
pm2 save
pm2 startup
```

Perintah terakhir mencetak satu baris lagi yang harus kamu jalankan. Tanpa itu, Palwise tidak hidup lagi setelah server di-reboot.

Jangan pernah menjalankan `npm run dev` di server.

6. Pasang Caddy

Ganti nama domain di `Caddyfile`, lalu:

```
sudo caddy start --config Caddyfile
```

HTTPS diurus Caddy sendiri, termasuk memperbarui sertifikatnya. Tidak ada yang perlu diingat tiap tiga bulan.

Porta 4000 sengaja TIDAK diteruskan ke internet. Cukup dashboard yang menghubunginya, dan itu terjadi di dalam server.

7. Beri tahu Midtrans ke mana harus mengabari

Di dashboard Midtrans, menu **Settings > Configuration**, isi **Payment Notification URL**:

```
https://app.palwise.id/api/pembayaran/midtrans
```

Lalu di menu **Settings > Snap Preferences**, isi bagian Redirection Settings. Bawaannya

`http://example.com`, dan itu bukan alamat kosong, itu alamat orang lain.

Kolom	Isi
Finish URL	<code>https://app.palwise.id/app/tagihan?bayar=selesai</code>
Error Payment URL	<code>https://app.palwise.id/app/tagihan?bayar=gagal</code>

Dua alamat itu cuma cadangan: kode kita mengirim `callbacks.finish` sendiri di setiap transaksi, dan yang per-transaksi menang. Tetap diisi supaya kalau suatu hari ada transaksi yang dibuat tanpa callback, orangnya tidak mendarat di situs contoh milik orang lain.

Setelan lain di halaman yang sama:

- **Language: Indonesian.** Yang membaca pemilik toko, bukan orang IT.
- **Split Midtrans fee with customer: MATIKAN.** Ini yang paling berbahaya di seluruh halaman itu. Kalau dinyalakan, biaya transaksi ditambahkan ke jumlah yang dibayar pelanggan, jadi jumlah yang dilaporkan Midtrans tidak lagi sama dengan yang kita catat. Pemeriksaan jumlah di webhook akan

menolak SETIAP pembayaran yang sah: uangnya masuk ke rekeningmu, tapi paketnya tidak pernah naik dan pelanggannya tidak tahu kenapa.

- **Hide Order Id: biarkan mati.** Kode transaksinya huruf acak dan tidak berarti apa-apa bagi pelanggan, tapi dia satu-satunya pegangan waktu ada yang bilang "uangnya sudah keluar tapi paketnya belum naik".
- **Custom expiry: biarkan Default.** Kalau diubah jadi pendek, tombol "Lanjutkan pembayaran" di halaman Paket akan menawarkan tautan yang sudah mati, karena `JAM_UPAYA_BAYAR_KEDALUWARSA` di `packages/db/src/langganan.ts` menganggap tautannya hidup 24 jam. Ubah dua-duanya kalau memang mau diubah.

Ini langkah yang paling gampang terlewat dan paling mahal kalau terlewat. Halaman "terima kasih" sesudah bayar cuma bukti browser-nya sampai ke situ, bukan bukti uangnya masuk. Yang menaikkan paket cuma notifikasi dari Midtrans ke alamat ini. Tanpa itu, uangnya masuk ke rekeningmu, paketnya tidak pernah naik, dan tidak ada satu pun pesan galat yang muncul di mana pun. Pelangganmu yang memberitahumu, dan dia sudah kesal waktu melakukannya.

Sesudah HTTPS-nya hidup, buktikan alurnya sekali:

```
npm run uji:bayar
```

Dia memalsukan sembilan notifikasi ke alamat itu, termasuk tanda tangan palsu dan jumlah yang tidak cocok, memakai server key dari `.env`. Kalau semuanya lolos, webhook-nya menerima yang sungguhan dan menolak yang palsu. Dia membuat satu workspace bernama "Uji Bayar" lalu menghapusnya lagi.

8. Akun bantuan Palwise

```
npm run akun:bantuan
```

Menyiapkan akun `bantuan@palwise.id` lengkap dengan asisten dan info produknya. Catat sandinya, masuk, lalu scan QR di halaman Nomor WhatsApp. Sesudah nomornya tersambung, isi nomor itu ke `IDENTITAS.waBantuan` di `apps/web/src/lib/identitas.ts` lalu bangun ulang.

9. Periksa sebelum diumumkan

Cek	Cara
Dashboard hidup	Buka <code>https://app.palwise.id</code> , harus dilempar ke halaman masuk
Halaman jualan hidup	Buka <code>https://palwise.id</code>
Pengalihan jalan	Buka <code>https://palwise.id/app</code> , harus pindah ke <code>app.palwise.id</code>
robots.txt ada	Buka <code>https://palwise.id/robots.txt</code>
Kartu bagikan muncul	Kirim tautannya ke WhatsApp sendiri, harus ada gambar
Email jalan	Daftar akun baru, konfirmasinya harus masuk
Lupa password jalan	Minta tautan, harus sampai dalam hitungan menit
Worker sehat	<code>pm2 logs palwise-worker</code> , tidak ada galat berulang
Asisten menjawab	Halaman Coba dulu, tanya soal harga
Webhook bayar jalan	<code>npm run uji:bayar</code> , semuanya harus lolos
Penjaga langganan hidup	<code>pm2 logs palwise-worker</code> , cari "penjaga langganan aktif"
Mutu jawaban modelnya	<code>npm run uji:model</code> , 8 uji ke API sungguhan
Panduan kebuka	Buka <code>https://palwise.id/panduan</code>
Halaman founder tertutup	Buka <code>/app/founder</code> dari akun lain, harus 404
Porta 4000 tertutup	Dari laptop: <code>curl http://IP-SERVER:4000</code> harus gagal

Merawat

Cadangan database. `npm run db:backup` menyalin `data/palwise.db` ke `data/cadangan` . Pasang di cron harian, dan **salin keluar dari server itu**. Cadangan yang tinggal di mesin yang sama tidak menolong waktu mesinnya yang hilang.

```
0 2 * * * cd /path/palwise && npm run db:backup
```

Lihat catatan proses: `pm2 logs` .

Perbarui kodenya:


```
cd /opt/palwise && git pull && npm install && npm run db:push && npm run build && pm2 restart all
```

`.env` dan `data/` tidak ikut tersentuh `git pull`, jadi kunci dan database di server tetap utuh setiap kali diperbarui.

Kalau `git pull` menolak karena ada berkas yang berubah di server, jangan langsung `git checkout .` — periksa dulu apa yang berubah dengan `git status`. Berkas yang berubah di server biasanya berarti ada yang menyunting langsung di sana, dan membuangnya berarti membuang perbaikan yang mungkin belum ada di laptop.

`pm2 restart all` memutus semua sambungan WhatsApp sebentar dan menyambung lagi sendiri. Lakukan di jam sepi.

Kalau `.env` di server perlu diubah, ingat mana yang butuh build ulang bukan cuma restart.

`AUTH_SECRET`, `DATABASE_URL`, `NEXT_PUBLIC_SITE_URL`, `NEXT_PUBLIC_APP_URL`, `MIDTRANS_SERVER_KEY`, dan `FOUNDER_EMAILS` ditanam ke dalam hasil build lewat blok `env` di `apps/web/next.config.mjs`.

Mengubahnya lalu cuma `pm2 restart` berarti nilai lamanya masih berlaku, dan tidak ada galat apa pun yang memberi tahu.

Yang cuma butuh restart worker: `GEMINI_MODEL`, `GEMINI_FALLBACK_MODEL`, dan sisanya yang cuma dibaca worker.

Yang paling sering salah

Nomor WhatsApp keluar sendiri berulang kali. Hampir selalu karena ada dua proses memegang sesi yang sama. Pastikan tidak ada `npm run dev` yang tertinggal jalan, dan PM2 cuma punya satu `palwise-worker`.

Halaman muncul tapi tidak ada tombol yang berfungsi. Berkas hasil build-nya rusak, biasanya karena `npm run build` dijalankan sementara aplikasinya masih hidup. Hentikan, hapus `apps/web/.next`, bangun ulang, nyalakan lagi.

Email tidak pernah sampai. Cek `RESEND_API_KEY`, dan cek domain pengirim di `EMAIL_FROM` sudah diverifikasi di Resend.

Worker menolak jalan. Baca pesannya. Kalau menyebut `INTERNAL_TOKEN`, itu memang disengaja: tokennya masih nilai bawaan yang tertulis di dalam kode.

Uangnya masuk tapi paketnya tidak naik. Hampir selalu Payment Notification URL di dashboard Midtrans belum diisi, atau salah alamat. Cek `pm2 logs palwise-web`: kalau webhook-nya benar-benar dipanggil, ada satu baris "Pembayaran ... lunas". Kalau tidak ada baris apa pun, Midtrans tidak pernah menghubungi servermu.

Halaman bayar gagal dibuka. Baca kalimatnya di halaman Paket & pemakaian. Kalau dia menyebut kunci salah lingkungan, `MIDTRANS_SERVER_KEY` dan `MIDTRANS_PRODUCTION` tidak sepasang. Kunci sandbox

berawalan SB- .

Langganan yang habis tidak turun sendiri. Yang menurunkannya worker, bukan dashboard. Kalau worker mati, paket berbayar yang kedaluwarsa tetap berlaku, dan halaman tagihan akan menampilkan tanggal yang sudah lewat.